18

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

커스텀 프로바이더와 인젝션 토큰

제2부 NestJS 핵심 · 5장 의존성 주입

쓰는 쪽 코드는 그대로, 들어가는 객체만 환경에 따라 교체



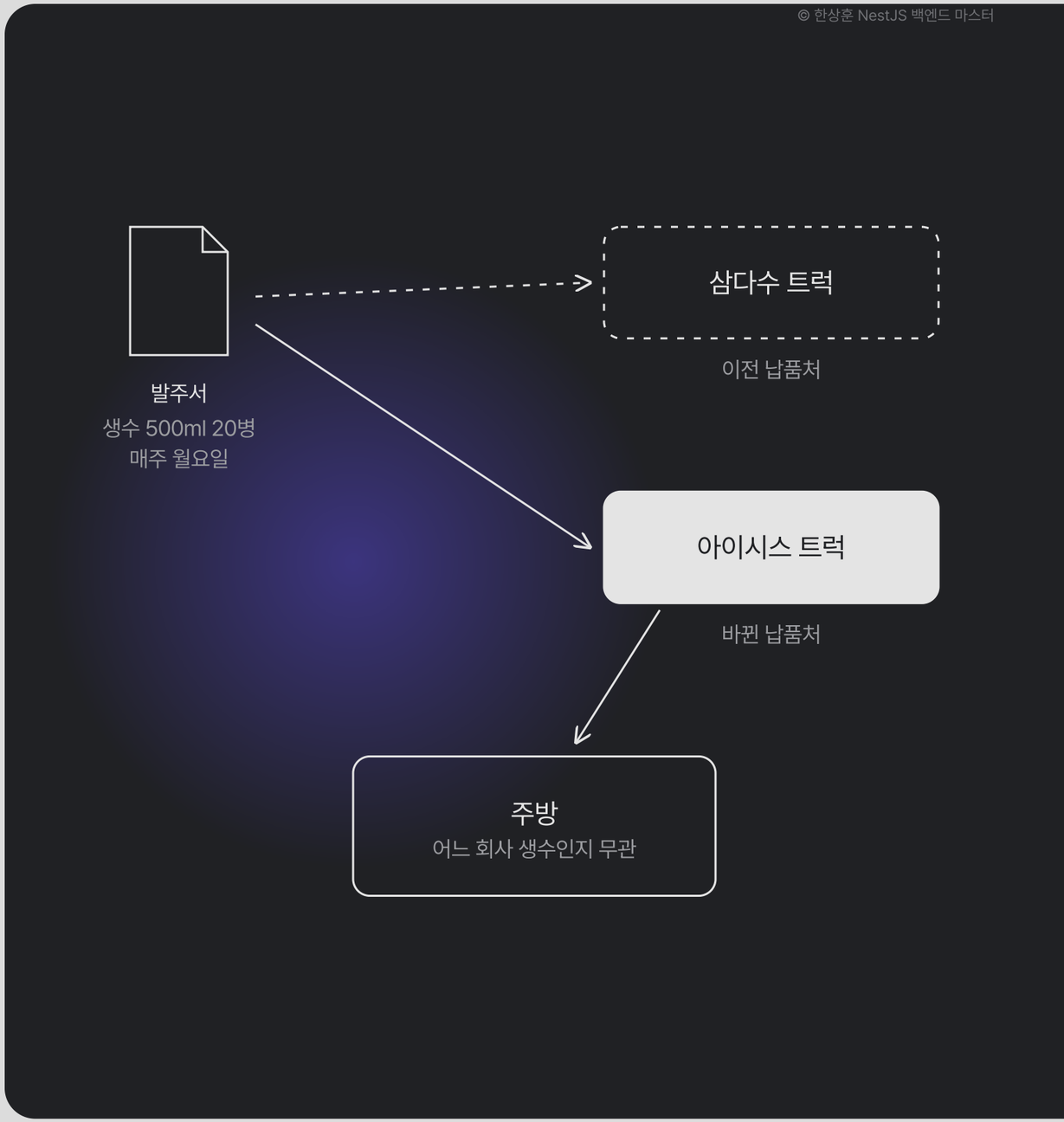
© 한상훈 NestJS 백엔드 마스터

생수 20병 발주서

→ 발주서: 생수 500ml 20병, 매주 월요일

→ 납품처: 삼다수 → 아이시스 교체

→ 주방: 발주서 그대로, 어느 회사 생수인지 무관



© 한상훈 NestJS 백엔드 마스터

환경마다 다른 결제사

환경	넣고 싶은 결제사	이유
로컬 개발	가짜	결제사 계정 없이 개발
테스트·CI	가짜	빠르고 결과가 일정
운영	진짜(HTTP)	실제 결제

→ 쓰는 쪽(예매·환불 서비스): 어느 환경이든 같은 코드

© 한상훈 NestJS 백엔드 마스터

인터페이스로 받는 결제사

```ts
// payments/payment-port.ts
export interface PaymentPort {
  charge(amount: number): Promise<{ approvalId: string; amount: number }>;
}

// reservations/reservations.service.ts
import type { PaymentPort } from '../payments/payment-port.js';

@Injectable()
export class ReservationsService {
  constructor(private readonly gateway: PaymentPort) {}
```

© 한상훈 NestJS 백엔드 마스터

인터페이스가 남긴 Object

dist/apps/ticketing/main.js

```
ReservationsService = _ts_decorate([
    (0,_nestjs_common__rspack_import_0.Injectable)(),
    _ts_metadata("design:type", Function),
    _ts_metadata("design:paramtypes", [
        typeof PaymentPort === "undefined" ? Object : PaymentPort
    ])
], ReservationsService);
```

```
ERROR [ExceptionHandler] UnknownDependenciesException [Error]: Nest can't resolve dependencies of the
ReservationsService (?). Please make sure that the argument at index [0] is available in the current module.
```

→ 인터페이스: 타입 전용, 빌드 후 삭제

© 한상훈 NestJS 백엔드 마스터

인젝션 토큰

→ 인젝션 토큰(injection token): Nest가 프로바이더를 찾는 열쇠

→ 조건: 실행 중에 남는 값

→ 클래스(추상 클래스 포함), 심벌, 문자열

→ 클래스를 `providers`에 넣으면: 그 클래스 자체가 토큰

© 한상훈 NestJS 백엔드 마스터

추상 클래스 PaymentGateway

payment-gateway.ts

```ts
export interface Payment {
  approvalId: string;
  amount: number;
}

export abstract class PaymentGateway {
  abstract charge(amount: number): Promise<Payment>;
  abstract refund(approvalId: string): Promise<{ refunded: string }>;
}
```

→ 추상 클래스: 빌드 후에도 남는 클래스 → 토큰 가능

→ 생성자 타입 그대로: PaymentGateway

© 한상훈 NestJS 백엔드 마스터

가짜와 진짜

```
@Injectable()
export class FakePaymentGateway extends PaymentGateway {
  async charge(amount: number) {
    return { approvalId: `fake-${amount}`, amount };
  }
}

@Injectable()
export class HttpPaymentGateway extends PaymentGateway {
  async charge(amount: number) {
    const res = await fetch(`${this.options.baseUrl}/charge`, {
      method: 'POST',
      body: JSON.stringify({ amount }),
      signal: AbortSignal.timeout(this.options.timeoutMs),
    });
    const { approvalId } = await res.json();
    return { approvalId, amount };
  }
}
```

→ 생성 로그, refund는 생략 (부록)

© 한상훈 NestJS 백엔드 마스터

useClass로 고르기

payments.module.ts

```ts
@Module({
  providers: [
    {
      provide: PaymentGateway,
      useClass:
        process.env.PAYMENT_MODE === 'fake' ? FakePaymentGateway : HttpPaymentGateway,
    },
  ],
  exports: [PaymentGateway],
})
export class PaymentsModule {}
```

→ provide: 토큰 / useClass: 실제로 만들 클래스

© 한상훈 NestJS 백엔드 마스터

같은 코드, 다른 결제사

터미널

```
$ PAYMENT_MODE=fake node dist/apps/ticketing/main.js
FakePaymentGateway #1 생성
ReservationsService 생성
RefundsService 생성
{"seatId":"A-3-15","approvalId":"fake-110000","amount":110000}

$ PAYMENT_MODE=http node dist/apps/ticketing/main.js
HttpPaymentGateway 생성: http://localhost:5018
ReservationsService 생성
RefundsService 생성
{"seatId":"A-3-15","approvalId":"pg-110000","amount":110000}
```

→ 5018번: 로컬 가짜 결제사 서버 (`fake-pg.mjs`)

→ 예매·환불 서비스 코드: 변경 없음

© 한상훈 NestJS 백엔드 마스터

커스텀 프로바이더 네 가지

형태	넣는 것	쓰는 때
`useClass`	클래스 (Nest가 생성)	환경별 구현 교체
`useValue`	이미 있는 값	설정 객체, 테스트용 가짜
`useFactory`	만드는 함수	다른 값이 있어야 만들 수 있을 때, 비동기 준비
`useExisting`	다른 토큰	같은 객체에 이름 하나 더

→ 커스텀 프로바이더: provide(토큰) + 넣을 것을 직접 지정한 등록

© 한상훈 NestJS 백엔드 마스터

설정 값과 심벌 토큰

```
// payments/payment-options.ts
export const PAYMENT_OPTIONS = Symbol('PAYMENT_OPTIONS');

export interface PaymentOptions {
  baseUrl: string;
  timeoutMs: number;
}

// payments/payments.module.ts
{
  provide: PAYMENT_OPTIONS,
  useValue: { baseUrl: 'http://localhost:5018', timeoutMs: 3000 },
},
```

→ 설정 객체: 클래스가 없으므로 심벌을 토큰으로

→ Symbol: 같은 설명 문자열이어도 서로 다른 값

© 한상훈 NestJS 백엔드 마스터

@Inject로 받는 값

http-payment-gateway.ts

```
@Injectable()
export class HttpPaymentGateway extends PaymentGateway {
  constructor(@Inject(PAYMENT_OPTIONS) private readonly options: PaymentOptions) {
    super();
    console.log(`HttpPaymentGateway 생성: ${options.baseUrl}`);
  }
```

→ 타입 PaymentOptions: 인터페이스, 메타데이터로는 `Object`

→ `@Inject(토큰)`: 메타데이터 대신 이 토큰으로 검색

→ 테스트: `useValue`로 가짜 객체를 그대로 등록

© 한상훈 NestJS 백엔드 마스터

접속 토큰이 있어야 만드는 클라이언트

http-payment-gateway.ts

```
export class HttpPaymentGateway extends PaymentGateway {
  constructor(
    private readonly options: PaymentOptions,
    private readonly accessToken: string,
  ) {
    super();
    console.log(`HttpPaymentGateway 생성: 토큰 ${accessToken}`);
  }
```

→ 결제사 접속 토큰: 결제사 /token 호출로 발급

→ 생성자 인자에 문자열: Nest가 알아서 채울 수 없음

© 한상훈 NestJS 백엔드 마스터

useFactory와 inject

payments.module.ts

```
{
  provide: PaymentGateway,
  inject: [PAYMENT_OPTIONS],
  useFactory: async (options: PaymentOptions) => {
    if (process.env.PAYMENT_MODE === 'fake') return new FakePaymentGateway();
    const res = await fetch(`${options.baseUrl}/token`, { method: 'POST' });
    const { accessToken } = await res.json();
    return new HttpPaymentGateway(options, accessToken);
  },
},
```

→ `inject`: 팩토리 인자로 받을 토큰 목록, 순서대로

→ async 팩토리: Promise가 끝날 때까지 기다린 뒤 주입

© 한상훈 NestJS 백엔드 마스터

기다렸다가 만드는 순서

부팅 로그

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
SeatsService 생성
LOG [InstanceLoader] AppModule dependencies initialized
LOG [InstanceLoader] ConcertsModule dependencies initialized
LOG [InstanceLoader] PerformancesModule dependencies initialized
LOG [InstanceLoader] SeatsModule dependencies initialized
HttpPaymentGateway 생성: 토큰 tok-123
ReservationsService 생성
RefundsService 생성
LOG [InstanceLoader] PaymentsModule dependencies initialized
LOG [InstanceLoader] ReservationsModule dependencies initialized
LOG [NestApplication] Nest application successfully started
```

→ 결제 모듈과 예매 모듈이 맨 뒤:
토큰 응답을 기다린 만큼

© 한상훈 NestJS 백엔드 마스터

답이 없는 결제사

```
// fake-pg.mjs: MODE=hang이면 응답하지 않음
http.createServer((req, res) => {
  if (mode === 'hang') return;
  ...
}).listen(Number(process.env.PORT ?? 5018));
```

→ 결제사 점검, 방화벽, 잘못된 주소: 접속은 되는데 응답이 없는 경우

© 한상훈 NestJS 백엔드 마스터

20초 뒤에도 안 열린 포트

터미널

```
$ MODE=hang node fake-pg.mjs &
$ PORT=4018 node dist/apps/ticketing/main.js
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
SeatsService 생성
LOG [InstanceLoader] AppModule dependencies initialized
LOG [InstanceLoader] ConcertsModule dependencies initialized
LOG [InstanceLoader] PerformancesModule dependencies initialized
LOG [InstanceLoader] SeatsModule dependencies initialized
(20초 동안 다음 줄 없음)

$ curl localhost:4018/
curl exit 7 (연결 거부)
```

→ 에러도 종료도 없음: 떠 있는 프로세스, 닫힌 포트

데이터 블록: 20초 동안 SeatsModule 줄 뒤 출력 없음, curl exit 7, 2026-10-26 강사 PC 실측, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

팩토리에도 시간 제한

```
const res = await fetch(`${options.baseUrl}/token`, {
  method: 'POST',
  signal: AbortSignal.timeout(options.timeoutMs),
});
```

```
LOG [InstanceLoader] SeatsModule dependencies initialized   (오후 2:12:19)
ERROR [ExceptionHandler] DOMException [TimeoutError]: The operation was aborted due to timeout   (오후 2:12:22)
exit 1
```

→ 3초 뒤 에러와 함께 종료

→ 배포 도구: 죽은 프로세스는 바로 감지, 멈춘 프로세스는 늦게 감지

데이터 블록: timeoutMs 3000, 2:12:19 → TimeoutError 2:12:22, exit 1, 2026-10-26 강사 PC 실측, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

옛 이름으로 받는 환불 서비스

refunds.service.ts

```
@Injectable()
export class RefundsService {
  constructor(@Inject('LEGACY_PAYMENT') private readonly gateway: PaymentGateway) {
    console.log('RefundsService 생성');
  }
```

→ 'LEGACY_PAYMENT': 결제 모듈을 고치기 전의 문자열 토큰이라고 가정

→ 쓰는 곳을 한 번에 못 고칠 때: 옛 토큰도 같이 등록

© 한상훈 NestJS 백엔드 마스터

useClass 별칭: 객체 둘

```
{ provide: 'LEGACY_PAYMENT', useClass: FakePaymentGateway },
```

부팅 로그

```
FakePaymentGateway #1 생성
FakePaymentGateway #2 생성
RefundsService 생성
ReservationsService 생성
```

→ 예매는 #1, 환불은 #2

→ 토큰 캐시, 커넥션 풀을 가진 객체면 둘이 따로 동작

© 한상훈 NestJS 백엔드 마스터

useExisting 별칭: 객체 하나

```
{ provide: 'LEGACY_PAYMENT', useExisting: PaymentGateway },
```

부팅 로그

```
FakePaymentGateway #1 생성
ReservationsService 생성
RefundsService 생성
```

→ useExisting: 이미 등록된 토큰의 객체를 다른 토큰으로도 제공

→ exports에 'LEGACY_PAYMENT'도 추가

© 한상훈 NestJS 백엔드 마스터

문자열 토큰 오타

```
constructor(@Inject('LEGACY_PAYMENTS') private readonly gateway: PaymentGateway) {
```

○ ○ ○ 터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
tsc exit 0

ERROR [ExceptionHandler] UnknownDependenciesException [Error]: Nest can't resolve dependencies of the
RefundsService (?). Please make sure that the argument "LEGACY_PAYMENTS" at index [0] is available in the
ReservationsModule module.
```

→ 문자열은 tsc가 검사하지 않음

→ 두 모듈이 같은 문자열을 쓰면 충돌

© 한상훈 NestJS 백엔드 마스터

심벌 상수 오타

```
inject: [PAYMENT_OPTION],
```

터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
apps/ticketing/src/payments/payments.module.ts(15,16): error TS2552: Cannot find name 'PAYMENT_OPTION'.
Did you mean 'PAYMENT_OPTIONS'?
```

→ 상수 이름 오타: 편집기와 tsc가 바로 표시

→ Symbol: 다른 파일의 같은 설명 문자열과도 충돌 없음

© 한상훈 NestJS 백엔드 마스터

토큰의 종류

토큰	예	오타 발견
클래스(추상 클래스)	`PaymentGateway`	tsc
심벌 상수	`PAYMENT_OPTIONS`	tsc
문자열	`'LEGACY_PAYMENT'`	부팅 때
인터페이스	`PaymentPort`	토큰 불가

© 한상훈 NestJS 백엔드 마스터

토큰과 프로바이더 등록

→ 예매·환불 서비스: `PaymentGateway`, `PAYMENT_OPTIONS`만 앎

→ 가짜·진짜, 설정 값, 별칭: PaymentsModule providers에서 결정

"쓰는 쪽은 토큰만 알고, 무엇을 넣을지는 프로바이더 등록이 정합니다."